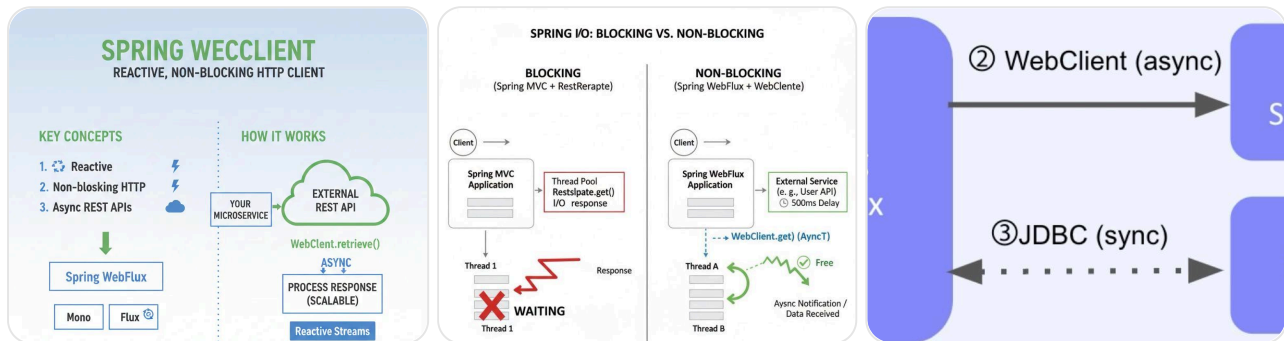


WebClient in Spring Boot



WebClient is Spring's non-blocking HTTP client used to communicate with other HTTP services/APIs.

In simple terms:

WebClient allows one Spring application to call another application's REST API over HTTP.

For example:

```

Order Service
  |
  | HTTP request
  ↓
WebClient
  |
  | GET /products/101
  ↓
Product Service
  |
  | HTTP response
  ↓
WebClient
  |
  ↓
Order Service
  
```

It belongs to **Spring WebFlux** and is designed around **reactive, non-blocking programming**.

1. Why do we need WebClient?

Suppose your `Order Service` needs product information from `Product Service`.

`Product Service` exposes:

```
</> http
```



Your `Order Service` needs to make an HTTP call.

You could use:

```
RestTemplate
```



or:

```
WebClient
```



With `WebClient`:

```
WebClient webClient = WebClient.create();

Product product = webClient
    .get()
    .uri("http://product-service/products/101")
    ...
```

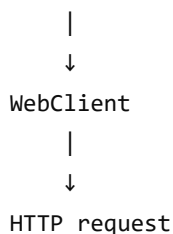
So `WebClient` acts as the **HTTP client** between your application and another HTTP server.

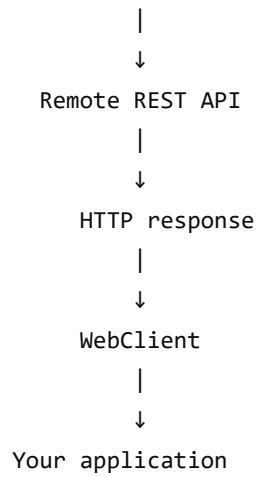
2. What exactly is WebClient?

`WebClient` is an interface/class provided by Spring WebFlux for making HTTP requests.

Conceptually:

Your Spring Boot application





It can make requests such as:

GET
POST
PUT
DELETE
PATCH



and handle:

Headers
Query parameters
Path variables
Request body
Response body
HTTP status codes
Timeouts
Errors
Authentication



3. WebClient is non-blocking

This is the most important characteristic of WebClient.

Suppose your application makes a request:

Order Service → Product Service



Product Service takes 2 seconds to respond.

With a traditional blocking HTTP client, the thread can remain blocked waiting for the response.

Conceptually:

Thread

```
|  
|---- send request  
|  
|    WAIT 2 seconds  
|  
|---- receive response  
|  
↓  
continue execution
```



WebClient is designed differently.

The request is initiated and the reactive framework can continue processing other work while waiting for the network response.

Conceptually:

Thread

```
|  
|---- send request  
|  
|---- do other work  
|  
|---- do other work  
|  
|---- response arrives  
|  
↓  
process response
```



This is why WebClient is called a **non-blocking HTTP client**.

4. WebClient is reactive

WebClient uses the **Reactive Streams** model.

The two important reactive types you will commonly see are:

Mono

</> Java



Mono

Represents:

0 or 1 result

For example:

```
GET /products/101
```



normally returns one product.

Therefore:

```
</> Java
```



is appropriate.

Flux

Represents:

0 to many results

For example:

```
GET /products
```



could return many products.

Therefore:

```
</> Java
```



can be used.

So:

```
One result      → Mono<T>  
Multiple results → Flux<T>
```



5. Basic WebClient structure

A typical WebClient call looks like:

```
webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```

Understand this as a sequence:

```
webClient
  ↓
.get()
  ↓
.uri(...)
  ↓
.retrieve()
  ↓
.bodyToMono(...)
```

Each part has a specific responsibility.

6. WebClient object

First you need a WebClient.

For example:

```
</> Java
```

This creates a WebClient with default configuration.

You can also provide a base URL:

```
WebClient webClient = WebClient.builder()
    .baseUrl("http://product-service")
```

```
</> Java
```

Now:

```
webClient  
    .get()
```

</> Java



effectively targets:

```
http://product-service/products/101
```



7. .get()

This specifies the HTTP method.

</> Java



means:

</> http



For example:

```
webClient  
    .get()
```

</> Java



Other methods include:

```
webClient.get()  
webClient.post()  
webClient.put()  
webClient.delete()  
webClient.patch()
```



So WebClient supports the common HTTP methods.

8. .uri()

`.uri()` specifies where the request should be sent.

Example:

```
</> Java
```



With:

```
</> Java
```



the final URL becomes:

```
http://product-service/products/101
```



You can also build URIs dynamically.

For example:

```
</> Java
```



If:

```
</> Java
```



the request becomes:

```
/products/101
```



9. .retrieve()

After specifying the request, you need to tell WebClient that you want to retrieve the response.

```
</> Java
```



Example:

```
webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono()
```



At this point, you are working with the HTTP response.

10. .bodyToMono()

Suppose Product Service returns:

```
{
  "id": 101,
  "name": "Laptop",
  "price": 75000
}
```



You want WebClient to convert this JSON into:

```
</> Java
```



You can use:

```
</> Java
```



Complete example:

```
Mono<Product> productMono = webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono()
```

```
.retrieve()
```

```
</> Java
```



Conceptually:

```
HTTP JSON
  ↓
WebClient
  ↓
Jackson conversion
  ↓
Product object
  ↓
Mono<Product>
```



11. Why does it return `Mono<Product>` instead of `Product` ?

This is very important.

You might initially expect:

```
Product product = webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```

But this isn't correct because `WebClient` is reactive.

It returns:

```
</> Java
```



because the result may arrive **later**.

Think of:

```
</> Java
```



as:

"A Product will be available asynchronously when the HTTP response arrives."

It isn't necessarily the Product itself at that moment.

12. WebClient does not automatically block

This is another important concept.

Consider:

```
Mono<Product> productMono = webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```

Creating this pipeline does not mean your code immediately receives the Product.

The reactive pipeline represents the operation.

The operation is executed as part of the reactive subscription process.

For example:

```
productMono.subscribe(product -> {
    System.out.println(product.getName());
});
```

</> Java

The callback runs when the result is available.

13. .block()

You can explicitly make a WebClient call blocking:

```
Product product = webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .block();
```

```
bodyToMono(Product.class)
```

</> Java



Now the current thread waits for the result.

Conceptually:

```
WebClient
  ↓
HTTP request
  ↓
WAIT
  ↓
response
  ↓
Product
```



Therefore:

WebClient itself is designed for non-blocking usage, but you can call `.block()` and make that particular operation blocking.

In a fully reactive application, you generally avoid unnecessary `.block()` calls.

14. Complete simple example

Suppose you have:

```
public class Product {

    private Long id;
    private String name;
    private double price;

    // getters/setters
}
```

You configure:

```
@Bean
WebClient webClient() {
    return WebClient.builder()
        .baseUrl("http://product-service")
```

```
.build();
```

```
</> Java
```



Then:

```
@Service
public class OrderService {

    private final WebClient webClient;

    public OrderService(WebClient webClient) {
        this.webClient = webClient;
    }

    public Mono<Product> getProduct(Long id) {

        return webClient
            .get()
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Product.class);
    }
}
```

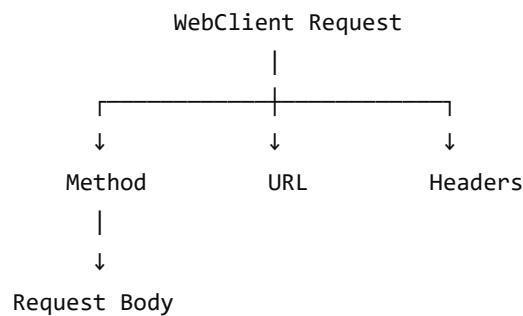
Flow:

```
OrderService
  |
  ↓
WebClient
  |
  ↓
GET /products/101
  |
  ↓
Product Service
  |
  ↓
JSON response
  |
  ↓
Jackson
  |
  ↓
Product
  |
  ↓
Mono<Product>
```



15. WebClient request components

A WebClient request can contain several things:



For example:

```
POST /orders/101?notify=true
Authorization: Bearer xyz
Content-Type: application/json

{
  "productId": 10,
  "quantity": 2
}
```

WebClient can construct all of these parts.

16. Request body

For a POST request:

```
webClient
    .post()
    .uri("/orders")
    .bodyValue(order)
    .retrieve()
    .bodyToMono(OrderResponse.class);
```

Here:

</> Java

means:

Put the `order` object into the HTTP request body.

Conceptually:

```
Java Order object
    ↓
Jackson
    ↓
JSON
    ↓
HTTP request body
```



17. Headers

You can add headers:

```
webClient
    .get()
    .uri("/products/101")
    .header("Authorization", "Bearer token")
    .retrieve()
    .bodyToMono(Product.class);
```

You can also use:

```
.headers(headers -> {
    headers.set("Authorization", "Bearer token");
```

</> Java



Headers are commonly used for:

```
Authorization
Content-Type
Accept
Correlation ID
Custom application headers
```



18. Query parameters

Suppose the API is:

```
/products?category=mobile&page=1
```



You can construct it using:

```
webClient
    .get()
    .uri(uriBuilder -> uriBuilder
        .path("/products")
        .queryParams("category", "mobile")
        .queryParams("page", 1)
        .build())
    .retrieve()
    .bodyToFlux(Product.class);
```

19. Path variables

For:

```
/products/101
```



you can use:

```
</> Java
```



Or:

```
.uri(uriBuilder -> uriBuilder
    .path("/products/{id}")
```

```
</> Java
```



20. Response body

WebClient can convert the response into different forms.

For one object:

```
</> Java
```



For multiple objects:

```
</> Java
```



For raw String:

```
</> Java
```



For example:

```
Mono<String> response = webClient
    .get()
    .uri("/hello")
    .retrieve()
    .bodyToMono(String.class);
```



21. The important mental model

When you see:

```
webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```



read it as:

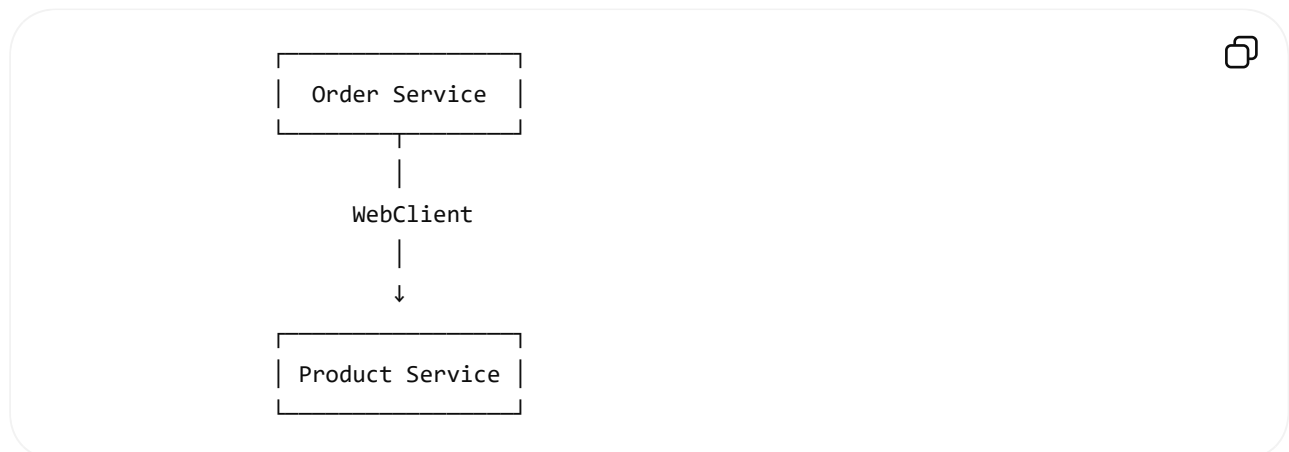
"Create a GET request to `/products/101` , retrieve the HTTP response, convert its response body into a `Product` , and represent that asynchronous result as a `Mono<Product>` ."

That's the core idea of WebClient.

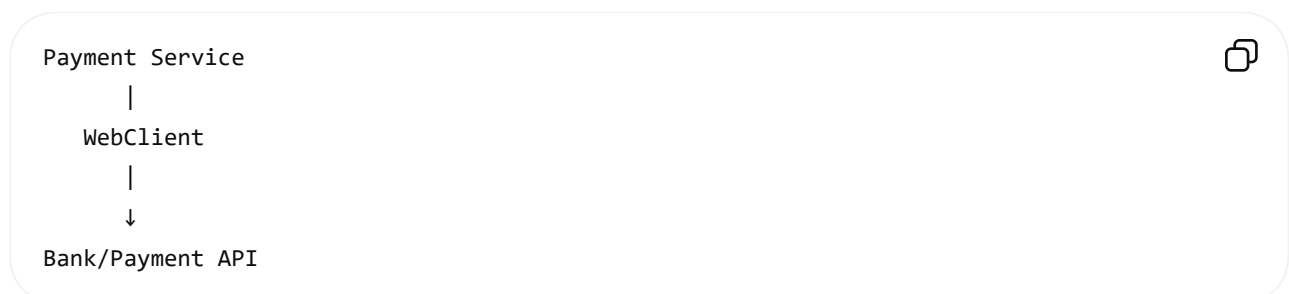
22. WebClient and microservices

WebClient is particularly useful in microservices because services frequently communicate over HTTP.

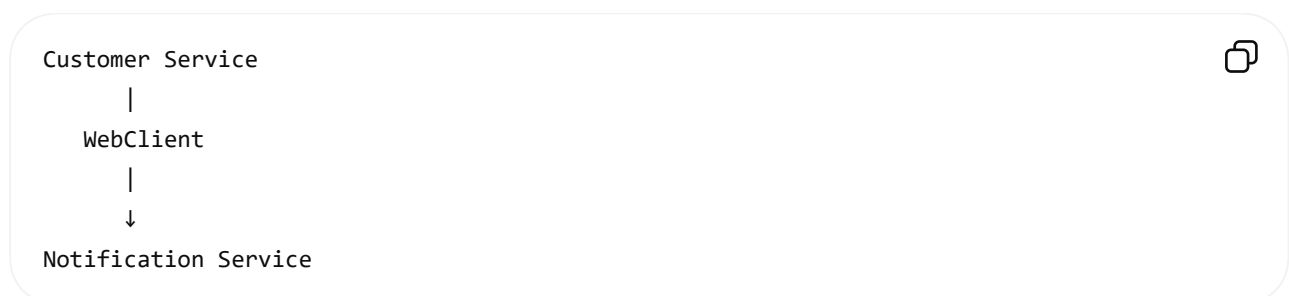
For example:



Another example:



Another:



So WebClient is an **HTTP-based inter-service communication mechanism**.

23. One important distinction

Don't think:

WebClient = REST API



They are different things.

REST API is an API design/style for exposing resources over HTTP.

WebClient is a client-side tool for making HTTP requests.

For example:

```
Product Service
  |
  | exposes REST API
  ↓
GET /products/101
```



Another service can consume that API using:

WebClient



So:

REST API = what the server exposes



WebClient = what the client uses to call it

24. WebClient's core purpose

The most important thing to remember:

WebClient is Spring WebFlux's reactive HTTP client used to make non-blocking HTTP requests and consume HTTP responses.

Its basic flow is:

```
Create WebClient
  ↓
Choose HTTP method
  ↓
Specify URI
  ↓
Add headers/body if required
  ↓
Send request
  ↓
```



```

Receive response
    ↓
Convert response body
    ↓
Mono / Flux

```

For interviews, the one-line definition to remember is:

WebClient is a non-blocking, reactive HTTP client provided by Spring WebFlux for making HTTP requests and communicating with external services or other microservices.



is this sufficient?

For understanding what WebClient is, yes.

But for a 5.5-year Java/Spring Boot interview, I would say **not yet**. You should cover these WebClient topics next:

1. WebClient architecture and request/response flow

2. Creating and configuring WebClient

- `WebClient.create()`
- `WebClient.builder()`
- `baseUrl()`
- default headers

3. HTTP methods — GET, POST, PUT, DELETE

4. URI

- path variables
- query parameters

5. Request body

- `bodyValue()`
- `body()`

6. Headers and authentication

7. Response handling

- `retrieve()`
- `bodyToMono()`
- `bodyToFlux()`
- `toEntity()`

8. HTTP status handling
 - `onStatus()`
9. Error handling
10. Timeouts
11. Retry
12. Mono vs Flux in WebClient
13. Blocking vs non-blocking
 - especially `.block()` and why it can be problematic
14. WebClient with Spring MVC vs Spring WebFlux
15. WebClient in microservice communication
16. Connection pooling / underlying HTTP client
17. Logging and debugging WebClient requests
18. Production configuration considerations

For your interview preparation

The most important areas are:

WebClient → request construction → `retrieve()` → response conversion → error handling → timeout → retry → blocking vs non-blocking → WebFlux/MVC usage.

So I would **not move to another topic yet**. We can go through WebClient from **beginner** → **interview level**, one topic at a time, without mixing in unrelated concepts.



explain each in detail

Yes. Let's cover **WebClient end-to-end**, topic by topic, while keeping the scope strictly to WebClient.

1. WebClient architecture and request/response flow

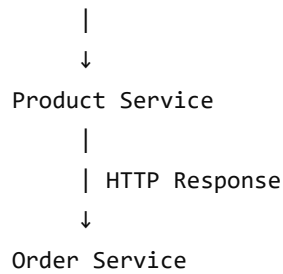
`WebClient` is a **reactive, non-blocking HTTP client** from Spring WebFlux.

Its job is to allow your application to communicate with another HTTP service.

Example:

```
Order Service
|
| WebClient
```





Suppose Product Service exposes:

```
</> http
```



The Order Service can call it using:

```
webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```

The flow is:

1. Create WebClient
- ↓
2. Build HTTP request
- ↓
3. Send HTTP request
- ↓
4. Remote service processes request
- ↓
5. HTTP response comes back
- ↓
6. WebClient receives response
- ↓
7. Response body is decoded
- ↓
8. Result is represented as Mono/Flux



Important

WebClient itself doesn't mean "REST".

It is an **HTTP client**.

It can communicate with:

- REST APIs
- JSON APIs
- XML APIs
- external HTTP services
- other microservices

2. Creating and configuring WebClient

There are two common ways.

2.1 WebClient.create()

</> Java



You can then specify the complete URL:

```
webClient
    .get()
    .uri("http://localhost:8081/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```



This is simple but not ideal when you have many calls to the same service.

2.2 WebClient.builder()

More commonly, you configure a reusable WebClient:

```
WebClient webClient = WebClient.builder()
    .baseUrl("http://localhost:8081")
```

</> Java



Then:

```
webClient
    .get()
```

```
.uri("/products/101")  
.retrieve()
```

</> Java



The final URL becomes:

```
http://localhost:8081/products/101
```



2.3 Spring Boot Bean

Usually, you create it as a Spring bean:

```
@Configuration  
public class WebClientConfig {  
  
    @Bean  
    public WebClient webClient() {  
        return WebClient.builder()  
            .baseUrl("http://localhost:8081")  
            .build();  
    }  
}
```

Then inject it:

```
@Service  
public class ProductService {  
  
    private final WebClient webClient;  
  
    public ProductService(WebClient webClient) {  
        this.webClient = webClient;  
    }  
}
```

This gives you a reusable WebClient instance.

3. baseUrl()

Suppose your Product Service is:

```
http://product-service:8081
```



You can configure:

```
WebClient.builder()  
    .baseUrl("http://product-service:8081")
```

</> Java



Then:

```
</> Java  
  
.uri("/products/101")
```



means:

```
http://product-service:8081/products/101
```



This avoids repeating the host for every request.

4. Default headers

Suppose every request needs:

```
</> http  
  
Content-Type: application/json
```



You can configure:

```
</> Java  
  
WebClient.builder()  
    .defaultHeader(  
        HttpHeaders.CONTENT_TYPE,  
        MediaType.APPLICATION_JSON_VALUE  
    )  
    .build();
```



You can also set headers for an individual request:

```
</> Java
```



```
webClient
    .get()
    .uri("/products/101")
    .header("X-Correlation-ID", "12345")
    .retrieve()
    .bodyToMono(Product.class);
```

Difference

defaultHeader()



Applied by default to requests

header()



Applied to a particular request



5. HTTP methods

WebClient supports the normal HTTP methods.

GET

Used to retrieve data:

</> Java

```
webClient
    .get()
    .uri("/products/101")
```



POST

Used to create data:

</> Java

```
webClient
    .post()
    .uri("/products")
```



PUT

Used to update/replace data:

</> Java



```
webClient
    .put()
    .uri("/products/101")
```

DELETE

Used to delete:

</> Java



```
webClient
    .delete()
    .uri("/products/101")
```

PATCH

Used for partial updates:

</> Java



```
webClient
    .patch()
    .uri("/products/101")
```

The general structure is:

</> Java



```
webClient
    .HTTP_METHOD()
    .uri(...)
    ...
```

6. URI

The URI tells WebClient **where to send the request**.

There are three common things to understand.

6.1 Simple URI

</> Java



```
.uri("/products")
```

6.2 Path variable

Suppose:

```
/products/101
```

Use:

```
</> Java  
.uri("/products/{id}", 101)
```

Or:

```
</> Java  
.uri("/products/{id}", productId)
```

If:

```
</> Java  
productId = 101
```

the URI becomes:

```
/products/101
```

7. Query parameters

Suppose the API is:

```
/products?category=mobile&page=1
```

You can build it:

```
</> Java  
webClient  
    .get()
```

```
.uri(uriBuilder -> uriBuilder
    .path("/products")
    .queryParams("category", "mobile")
    .queryParams("page", 1)
    .build())
.retrieve()
.bodyToFlux(Product.class);
```

The resulting URL is:

```
/products?category=mobile&page=1
```



Path variable vs query parameter

```
/products/101
      ↑
    Path variable
```



while:

```
/products?category=mobile
      ↑
    Query parameter
```



8. Request body

For POST/PUT/PATCH requests, you frequently need to send data.

Suppose:

```
</> Java

Product product = new Product(
    101L,
    "Laptop",
    75000
);
```



You can send it:

```
</> Java

webClient
    .post()
    .uri("/products")
```



```
.bodyValue(product)
.retrieve()
.bodyToMono(Product.class);
```

bodyValue() tells WebClient:

Put this Java object into the HTTP request body.

If the server expects JSON, Spring's configured HTTP message codecs/Jackson can serialize the object to JSON.

Conceptually:

```
Java Object
  ↓
JSON serialization
  ↓
HTTP Request Body
```



For example:

```
</> JSON

{
  "id": 101,
  "name": "Laptop",
  "price": 75000
}
```



9. Headers and authentication

Headers carry metadata about the request.

For example:

```
</> http

Authorization: Bearer abc123
```



=

You can send it:



...

```
</> Java

webClient
  .get()
  .uri("/products/101")
```



```
.header(  
    HttpHeaders.AUTHORIZATION,  
    "Bearer " + token  
)  
.retrieve()  
.bodyToMono(Product.class);
```

Other common headers:

Content-Type
Accept
Authorization
X-Correlation-ID



For example:

</> Java



```
.header(  
    HttpHeaders.ACCEPT,  
    MediaType.APPLICATION_JSON_VALUE  
)
```

10. Response handling

Suppose Product Service returns:

</> JSON



```
{  
  "id": 101,  
  "name": "Laptop",  
  "price": 75000  
}
```

You want:

</> Java



Product

You can write:

</> Java



```
.bodyToMono(Product.class)
```

For multiple products:

```
</> Java  
  
.bodyToFlux(Product.class)
```



So:

```
One object  
  ↓  
Mono<Product>  
  
Many objects  
  ↓  
Flux<Product>
```



11. retrieve()

This is one of the most commonly used WebClient APIs.

Example:

```
</> Java  
  
webClient  
  .get()  
  .uri("/products/101")  
  .retrieve()  
  .bodyToMono(Product.class);
```



retrieve() provides a convenient way to access the response body.

For example:

```
</> Java  
  
.bodyToMono(Product.class)
```



means:

Extract the response body and decode it into Product .

12. bodyToMono()

Use:

</> Java



```
.bodyToMono(Product.class)
```

when you expect **zero or one** logical result.

Example:

</> Java



```
Mono<Product> product = webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```

You can also receive a String:

</> Java



```
Mono<String> response = webClient
    .get()
    .uri("/hello")
    .retrieve()
    .bodyToMono(String.class);
```

13. bodyToFlux()

Suppose the server returns:

</> JSON



```
[
  {
    "id": 101,
    "name": "Laptop"
  },
  {
    "id": 102,
    "name": "Phone"
  }
]
```

You can use:

</> Java



```
Flux<Product> products = webClient
    .get()
    .uri("/products")
    .retrieve()
    .bodyToFlux(Product.class);
```

So:

bodyToMono()



One result

bodyToFlux()



Multiple results



14. toEntity()

Sometimes you don't want only the response body.

You also want:

HTTP status

Headers

Body



For that you can use:

</> Java



```
Mono<ResponseEntity<Product>> response = webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .toEntity(Product.class);
```

Now you can access:

</> Java



```
response.getStatusCode()
response.getHeaders()
response.getBody()
```

Conceptually:

```
ResponseEntity
|
├── Status
├── Headers
└── Body
```



This is useful when response metadata matters.

15. HTTP status handling

Suppose Product Service returns:

```
</> http

404 NOT_FOUND
```



You may want to handle it differently.

WebClient provides:

```
</> Java

.onStatus(...)
```



Example:

```
</> Java

webClient
    .get()
    .uri("/products/{id}", id)
    .retrieve()
    .onStatus(
        status -> status.value() == 404,
        response -> Mono.error(
            new ProductNotFoundException("Product not found")
        )
    )
    .bodyToMono(Product.class);
```



The idea is:

```
HTTP response
↓
```



Check status



404?



Create appropriate exception

You can similarly handle categories of statuses.

16. Error handling

There are different types of failures.

HTTP errors

Examples:

400
401
403
404
500
503



These are responses from the remote server.

Network errors

For example:

Connection refused
DNS failure
Connection reset



Timeout

The remote service doesn't respond within the configured time.

These should be handled differently depending on your application.

For example:

`</> Java`



```
webClient
    .get()
    .uri("/products/{id}", id)
    .retrieve()
```

```
.onStatus(  
    status -> status.is4xxClientError(),  
    response -> Mono.error(  
        new RuntimeException("Client error")  
    )  
)  
.onStatus(  
    status -> status.is5xxServerError(),  
    response -> Mono.error(  
        new RuntimeException("Server error")  
    )  
)  
.bodyToMono(Product.class);
```

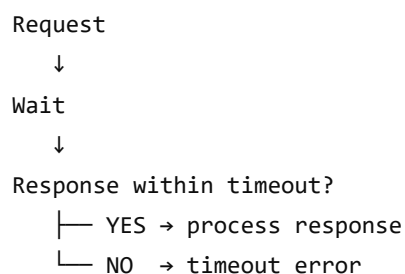
17. Timeout

Suppose Product Service normally responds within 500 ms.

You don't want your application waiting indefinitely.

A timeout can be configured so that the operation fails after a specified duration.

Conceptually:



Reactive timeout can be applied using Reactor operators, for example:

```
</> Java  
  
webClient  
    .get()  
    .uri("/products/101")  
    .retrieve()  
    .bodyToMono(Product.class)  
    .timeout(Duration.ofSeconds(2));
```

This means the reactive operation is allowed up to 2 seconds before timing out.

Important: reactive timeout and the underlying network client's connection/read/write timeout settings are related but not identical. Production applications often configure

both appropriately.

18. Retry

Suppose a temporary network problem occurs.

You may want to retry:

```
Attempt 1 → failure
Attempt 2 → failure
Attempt 3 → success
```



Reactor provides retry operators.

For example:

```
</> Java

webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class)
    .retry(3);
```



This can resubscribe after an error.

But **do not blindly retry every error**.

For example:

```
404 → retry usually doesn't help
400 → retry usually doesn't help
401 → retry usually doesn't help
temporary network failure → retry may help
503 → retry may sometimes help
```



Retries should generally be limited and carefully configured, especially for POST operations where repeating the request could have side effects.

19. Mono vs Flux in WebClient

This is essential.

Mono

```
</> Java
```



```
Mono<Product>
```

means:

```
0 or 1 Product
```



Example:

```
GET /products/101
```



Flux

```
</> Java
```



```
Flux<Product>
```

means:

```
0 to many Products
```



Example:

```
GET /products
```



Mental model:

```
Mono<T>
```

```
= one asynchronous result
```



```
Flux<T>
```

```
= sequence of asynchronous results
```

20. Blocking vs non-blocking

This is probably the most important interview topic.

WebClient is designed for **non-blocking I/O**.

Example:

</> Java



```
Mono<Product> productMono = webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class);
```

The reactive pipeline represents the operation without requiring the calling thread to wait synchronously for the response.

If you do:

</> Java



```
Product product = productMono.block();
```

you explicitly wait for the result.

Therefore:

WebClient



Non-blocking by design

.block()



Makes the calling code wait



.block() is not inherently forbidden, but you need to understand where you're using it.

In a reactive WebFlux application, unnecessary blocking can undermine the benefits of the reactive model and can cause scalability problems.

21. WebClient with Spring MVC

This is an important real-world point.

You can use WebClient even if your application uses traditional Spring MVC.

For example:

</> Java



```
@RestController
public class ProductController {
```

```
private final WebClient webClient;

...

@GetMapping("/products/{id}")
public Product getProduct(@PathVariable Long id) {

    return webClient
        .get()
        .uri("/products/{id}", id)
        .retrieve()
        .bodyToMono(Product.class)
        .block();
}
```

This works, but `.block()` makes the operation synchronous from the MVC controller's perspective.

22. WebClient with Spring WebFlux

In a reactive WebFlux application, you can keep the entire chain reactive:

</> Java



```
@GetMapping("/products/{id}")
public Mono<Product> getProduct(@PathVariable Long id) {

    return webClient
        .get()
        .uri("/products/{id}", id)
        .retrieve()
        .bodyToMono(Product.class);
}
```

Now:

Controller



Mono



WebClient



Remote service



No explicit blocking is necessary.

23. Connection pooling / underlying HTTP client

WebClient itself provides the higher-level HTTP API, while the actual network communication is handled through an underlying HTTP client/connector.

Depending on configuration, Spring WebFlux can use clients such as Reactor Netty.

Connection pooling allows connections to be reused.

Without reuse:

```
Request 1 → create connection → close  
Request 2 → create connection → close  
Request 3 → create connection → close
```



With connection pooling:

```
Create connection  
  ↓  
Reuse connection  
  ↓  
Reuse connection  
  ↓  
Reuse connection
```



This can reduce connection-establishment overhead.

In production, connection limits, idle timeouts, response timeouts, and other client settings may need appropriate configuration.

24. Logging and debugging WebClient

When a WebClient call fails, you often need to know:

```
Which URL?  
Which HTTP method?  
Which headers?  
Which status?  
How long did it take?  
What was the response?
```



You can add logging around the reactive chain:

</> Java



```
webClient
    .get()
    .uri("/products/101")
    .retrieve()
    .bodyToMono(Product.class)
    .doOnNext(product ->
        log.info("Received product {}", product.getId())
    )
    .doOnError(error ->
        log.error("Product API failed", error)
    );
```

For production systems, be careful not to log sensitive information such as:

Passwords
Access tokens
Authorization headers
Personal data
Sensitive request bodies



25. A complete WebClient example

Putting the major pieces together:

</> Java



```
@Service
public class ProductClient {

    private final WebClient webClient;

    public ProductClient(WebClient webClient) {
        this.webClient = webClient;
    }

    public Mono<Product> getProduct(Long id, String token) {

        return webClient
            .get()
            .uri("/products/{id}", id)
            .header(
                HttpHeaders.AUTHORIZATION,
                "Bearer " + token
            )
            .retrieve()
            .onStatus(
```

```

        status -> status.value() == 404,
        response -> Mono.error(
            new ProductNotFoundException(
                "Product not found"
            )
        )
    )
    .bodyToMono(Product.class)
    .timeout(Duration.ofSeconds(2))
    .doOnError(error ->
        log.error(
            "Failed to retrieve product {}",
            id,
            error
        )
    );
}

```

The flow is:

```

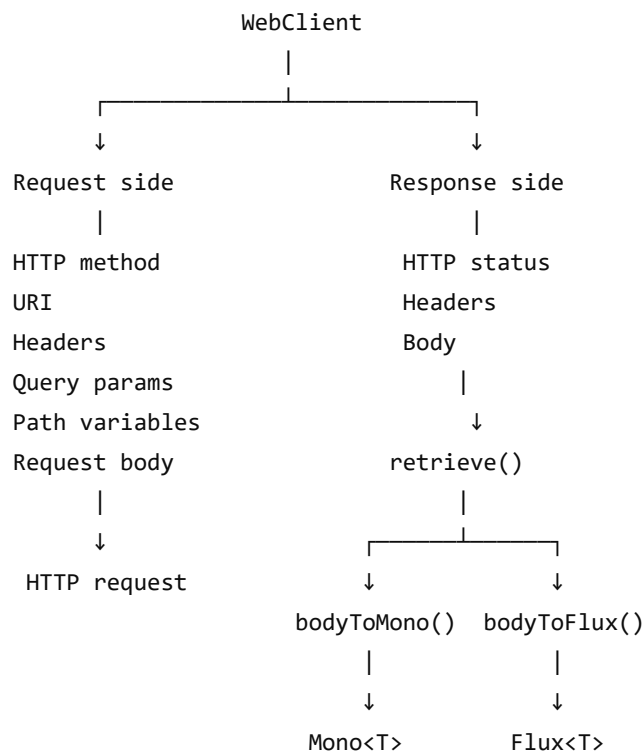
getProduct(101)
    ↓
WebClient
    ↓
GET /products/101
    ↓
Authorization header
    ↓
Product Service
    ↓
HTTP response
    ↓
Check HTTP status
    ↓
Convert JSON → Product
    ↓
Mono<Product>
    ↓
Timeout if necessary
    ↓
Handle/log error if necessary

```



26. The complete WebClient picture

For your 5.5-year Spring Boot interview, keep this mental structure:



And around this:

Timeout
 Retry
 Error handling
 Logging
 Authentication
 Connection management

The core interview definition

WebClient is a reactive, non-blocking HTTP client provided by Spring WebFlux. It is used to send HTTP requests to external APIs or other microservices and asynchronously process their responses using Reactor types such as `Mono` and `Flux`.

The most important APIs to remember

```

WebClient.builder()
  ↓
baseUrl()
  ↓
get() / post() / put() / delete()
  ↓
uri()
  ↓
header() / headers()
  ↓
bodyValue()
  
```

```
↓  
retrieve()  
↓  
onStatus()  
↓  
bodyToMono() / bodyToFlux()  
↓  
timeout()  
↓  
retry()
```

If you understand **why each of these exists and how the request flows through them**, you have the WebClient knowledge expected for most Spring Boot interviews at your experience level.



Files, images, and data analysis are unavailable until usage resets at 9:38 PM. Continue chatting with text only, or upgrade for more access.

✦ [Upgrade to Plus](#)

ChatGPT can make mistakes. Check important info.